

Python 프로그래밍

Assignment #1

소속: A반 1조

이름: 김남규

이메일: tkng00@naver.com

명예서약(Honor code)

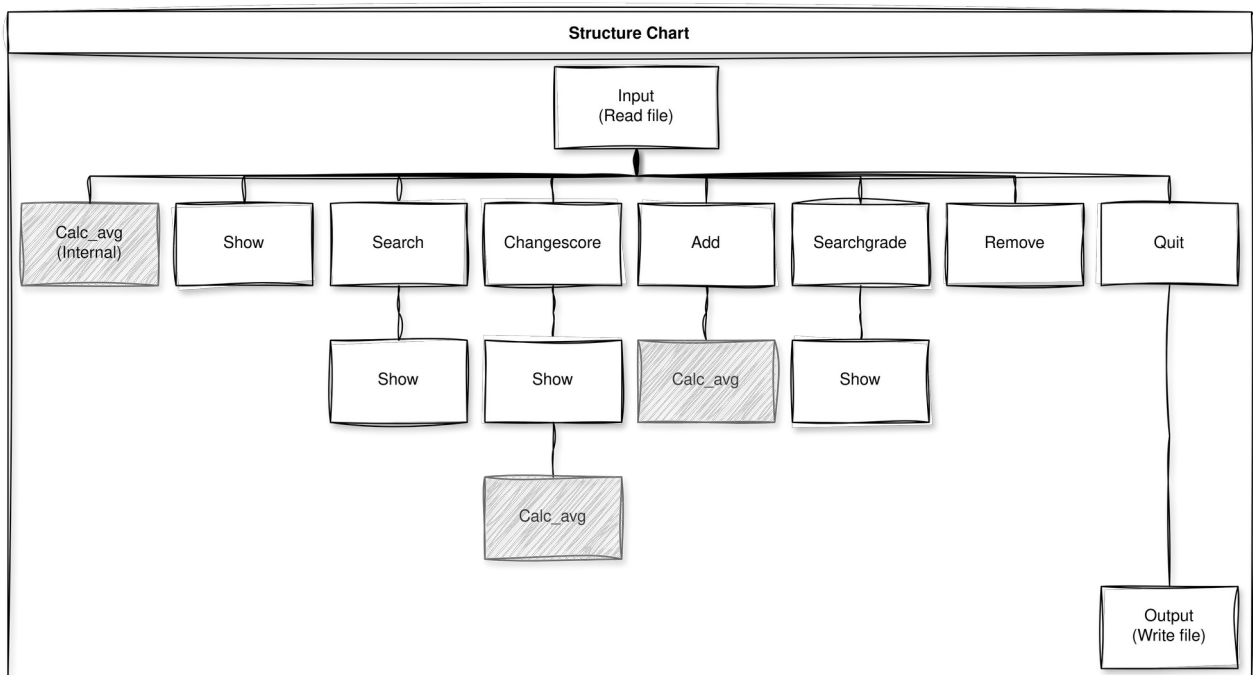
"나는 이 프로그래밍 과제를 다른 사람의 부적절한 도움 없이 완수하였습니다."

1. 개요

해당 과제는 파일로부터 성적 목록을 만들어 관리하는 성적 관리 프로그램이다.
본 프로그램을 간략하게 설명하면 다음과 같다.

- 파일로부터 데이터를 읽어 성적 목록을 만든다.
- 프로그램을 실행시켜 다음과 같은 역할을 구현한다.
 1. 학생 정보를 기반으로 평균과 등급을 산출한다.
 2. 전체 학생의 정보를 출력한다.
 3. 특정 학생의 학번을 검색하여 정보를 출력한다.
 4. 목록에 저장된 학생의 중간고사 혹은 기말고사 점수를 수정한다.
 5. 새로운 학생의 정보를 추가한다.
 6. 특정 등급을 입력받아, 그 등급에 해당하는 학생을 출력한다.
 7. 특정 학생의 정보를 삭제한다.
 8. 프로그램을 종료하고, 편집한 내용을 새로운 파일로 만든다.

이 때 사용되는 구조 차트(Structure chart)는 아래와 같이 표현할 수 있다.



- 입력부: 학생들의 성적 정보가 담겨 있는 파일을 열어 리스트로 만든다.
- 처리부: 사용자의 요청사항에 따라 정보를 추가 · 수정 · 삭제한다.
- 출력부: 변경된 정보를 저장하는 새로운 파일을 생성한다.

2. 프로그램 구조 및 설명

프로그램의 실행을 위한 알고리즘을 Pseudo 코드 형태로 나타내면 다음과 같다.

Pseudo algorithm for program
<pre>#1. 파일을 불러옴 if 파일명을 입력하지 않은경우, default값(students.txt)을 읽기모드로 불러옴. else 파일명을 입력한 경우, 파일명을 읽기 모드로 불러옴. #2. 데이터를 저장할 임시리스트를 구축 파일 안의 내용들을 읽어올 임시 리스트를 생성함. for 파일 안의 내용에 대해 한줄씩, 리스트에 내용을 담음. 열었던 파일을 닫음. #3. 데이터를 분리하여 리스트에 담음 성적을 처리할 리스트를 생성함. for 임시 리스트의 한줄씩에 대해, tab단위로 끊겨 있던 데이터를 분리해서 각각의 요소를 리스트에 담음. #4. 평균과 등급을 산출하여 리스트에 추가함 for 리스트 안의 들어있는 한 줄(리스트)씩, calc_avg함수를 활용하여 평균과 등급을 산출함. 리스트에 평균값을 추가함. 리스트에 등급을 추가함. #5. 프로그램을 실행함 while 중단되기 전까지 프로그램을 반복, 원하는 명령을 요구함 if 명령이 show인 경우, show함수를 시행함 elif 명령이 search인 경우, search함수를 시행함. elif 명령이 changescore인 경우, changescore함수를 시행함. elif 명령이 add인 경우, add함수를 시행함. elif 명령이 searchgrade인 경우, searchgrade함수를 시행함. elif 명령이 remove인 경우, remove함수를 시행함. elif 명령이 quit인 경우, quit함수를 시행함. 함수의 return값이 True인 경우, 반복을 종료함.(break) else 명령이 주어진 것들 중 없는 경우, 다시 반복으로 돌아감.(continue) "프로그램 끝"을 출력함</pre>

위의 Pseudo 코드는 가독성이 떨어지는 것을 막기 위하여 메인 함수의 작동 과정에 대해서만 정리하였다. 세부 기능을 수행하는 기타 함수는 알고리즘 부분에서 후술할 예정이다.

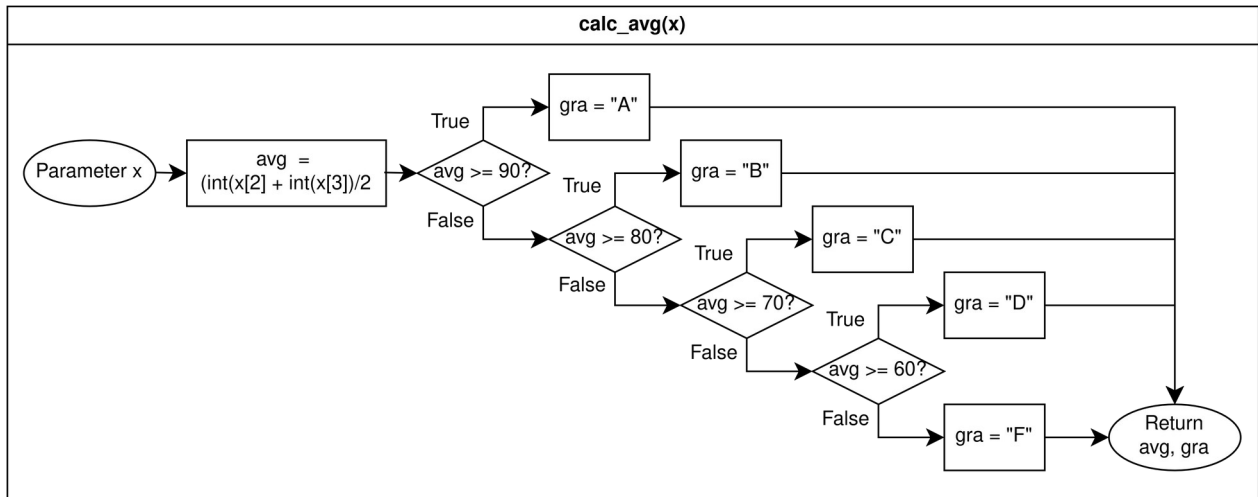
3. 세부 알고리즘

프로그램에서 실행되는 세부적인 함수들을 간단한 설명 및 Flow chart로 정리하였다.

1. calc_avg(x)

함수가 호출되면, 평균 계산을 수행한다.

이후 점수에 따라 등급을 부여하여 평균과 등급을 return한다.

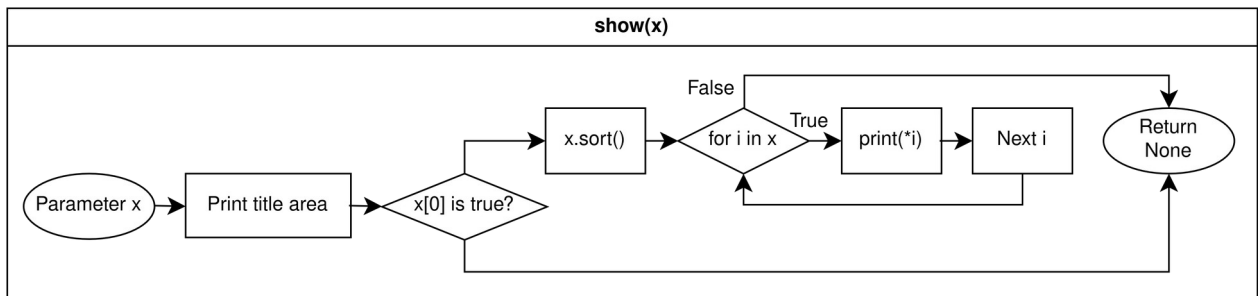


2. show(x)

함수가 호출되면, 표지부분을 출력한다.

이후 x값에 들어간 리스트가 비어있는지 확인한다.

리스트가 비어있지 않다면 평균에 맞춰 정렬한 후 리스트를 한개씩 출력한다.

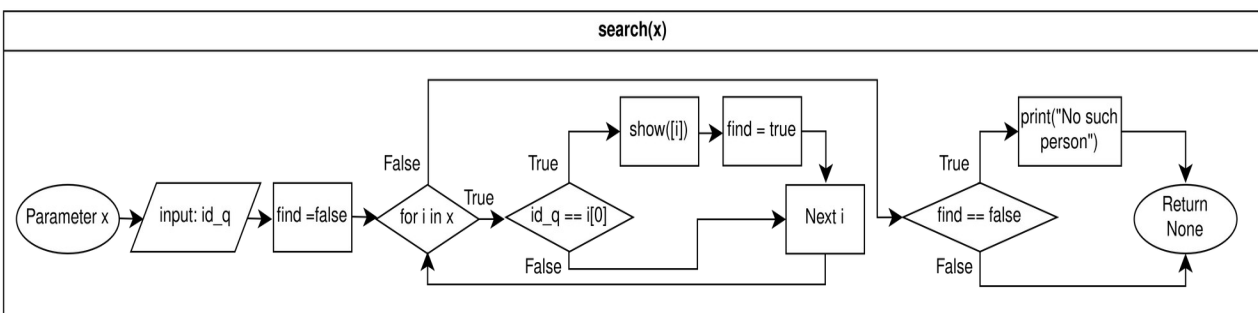


3. search(x)

함수가 호출되면 학번을 물어본다.

x안에 입력한 학번이 있다면, 해당 리스트의 값을 출력하고 검색 변수를 True로 바꾼다.

학번을 찾지 못했다면, "No such person"이라는 오류 메시지를 출력한다.



4. changescore(x)

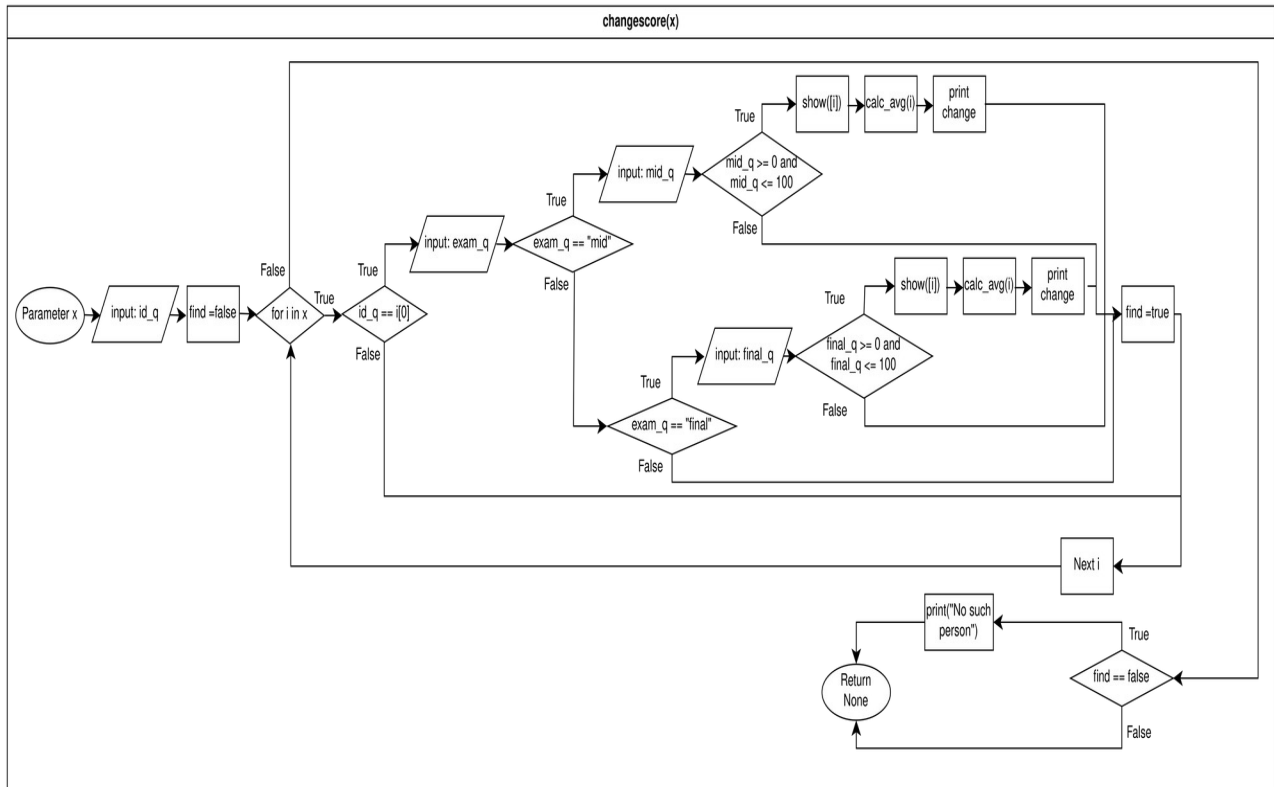
함수가 호출되면 학번을 물어본다.

x안에 입력한 학번이 있다면, 중간 혹은 기말인지를 물어본다.

중간 혹은 기말을 입력하였다면, 점수를 물어본다.

점수까지 올바르게 기입되었다면, 결과를 출력한다. 그리고 검색 변수를 True로 바꾼다.

학번을 찾지 못했다면, "No such person"이라는 오류 메시지를 출력한다.

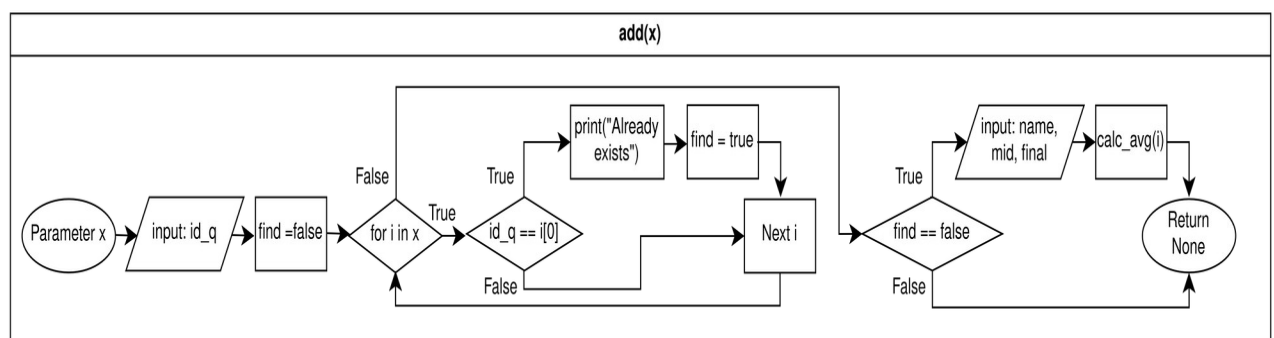


5. add(x)

함수가 호출되면 학번을 물어본다.

x안에 입력한 학번이 있다면, "Already exists"를 출력하고 검색변수를 True로 바꾼다.

학번을 찾지 못했다면, 이름과, 중간, 기말값을 물어보고 평균과 등급을 계산하여 추가한다.



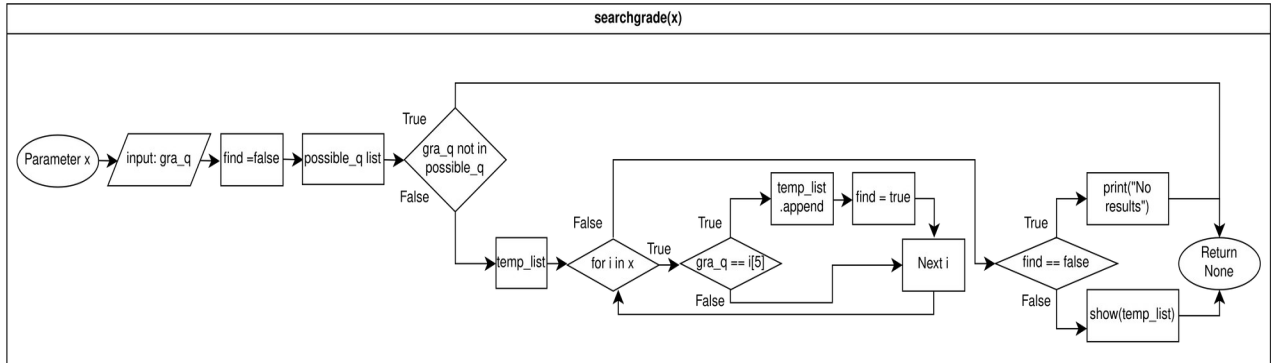
6. searchgrade(x)

함수가 호출되면 등급을 물어본다.

등급이 존재하는 등급이면, 해당 등급에 해당하는 학생을 찾는다.

학생이 존재하면, 학생의 리스트를 출력하고 검색 변수를 True로 바꾼다.

학생이 존재하지 않는다면 "No Results"를 출력한다.



7. remove(x)

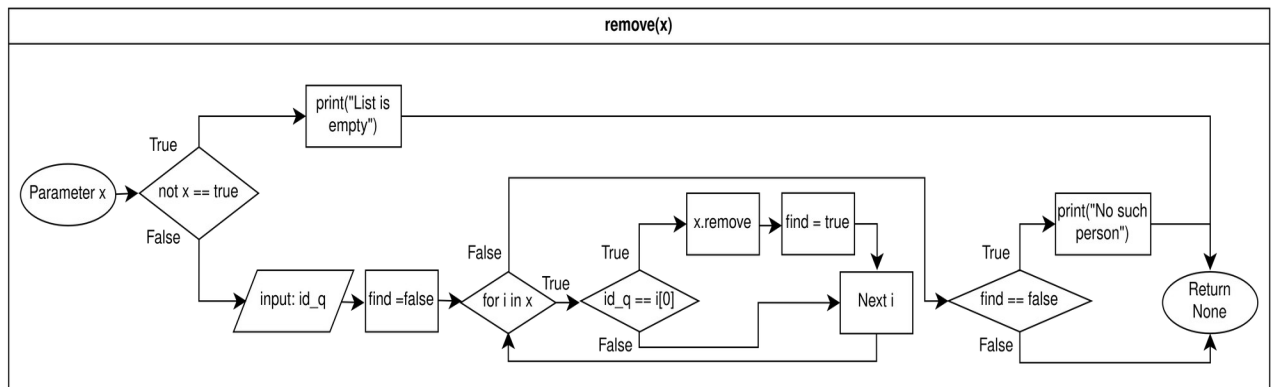
함수가 호출되면 parameter가 비어있는지 확인한다.

parameter로 받은 값이 비어 있다면 "List is empty"를 출력한다.

비어있지 않다면, 학번을 물어본다.

학생이 존재하면, 학생의 리스트를 삭제하고 검색 변수를 true로 바꾼다.

학생이 존재하지 않는다면 "No such person"을 출력한다.



8. quit(x)

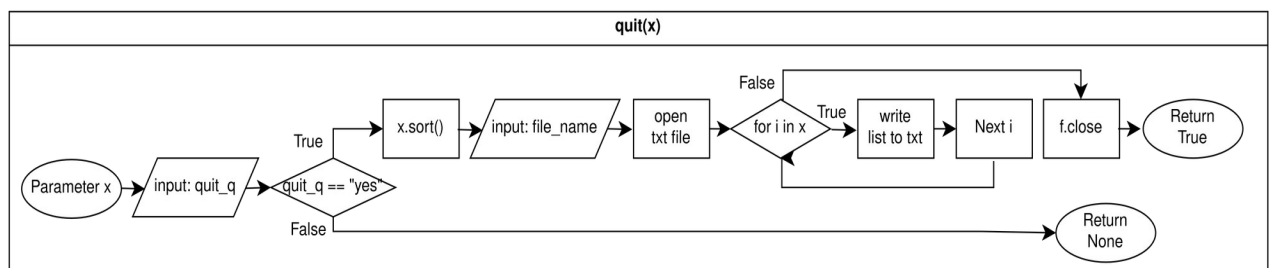
함수가 호출되면 저장 여부를 물어본다.

저장 여부가 yes라면 저장할 파일의 이름을 물어본다.

파일의 이름으로 연 txt파일로 x에 들어 있는 리스트들을 한줄씩 txt파일에 적는다.

다 적었다면 파일을 닫고 True를 return한다.

외의 값에 대해서는 None값을 return한다.



4. 프로그램 실행 결과 및 예제

터미널 환경에서 위 명령어를 통해 모듈을 시행하면 프로그램이 시작되고, 명령어 입력이 가능하다.

```
(base) pi1@pi1-Precision-7920-Tower:~/Desktop/git$ python project.py students.txt
#
```

정의된 명령어 이외의 잘못된 명령어를 입력하는 경우, 에러 메시지 없이 다음 명령어를 입력 받을 준비를 한다.

```
# find
#
```

show를 입력하는 경우, 전체 목록을 평균 점수를 기준으로 내림차순으로 출력한다.

```
# show
Student      Name      Mid      Final      Average Grade
-----
20180002     Lee Jieun    92        89        90.5      A
20180009     Lee Yeonghee 81        84        82.5      B
20180001     Hong Gildong 84        73        78.5      C
20180011     Ha Donghun   58        68        63.0      D
20180007     Kim Cheolsu  57        62        59.5      F
#
```

search를 입력하는 경우, 검색하고자 하는 학생의 학번을 요구한다. 찾고자 하는 학생이 목록에 없는 경우 "NO SUCH PERSON"이라는 에러 메시지를 출력한다.

```
# search
Student ID:20180050
No Such Person
#
```

학생이 있는 경우 학번, 이름, 중간고사 점수, 기말고사 점수, 평균, 학점을 출력한다.

```
# search
Student ID:20180002
Student      Name      Mid      Final      Average Grade
-----
20180002     Lee Jieun    92        89        90.5      A
```

changescore를 입력하는 경우, 검색하고자 하는 학생의 학번을 요구한다. 학생이 목록에 없는 경우 "NO SUCH PERSON"이라는 에러 메시지를 출력한다.

```
# changescore
Student ID:20180050
No Such Person
#
```

학생을 찾은 경우, 수정할 시험 종류를 요구한다. 잘못 입력한 경우 처음 명령으로 돌아간다.

```
# changescore
Student ID:20180007
Mid/Final?miid
#
```

시험 종류를 올바르게 입력하였다면, 시험 점수를 요구한다. 이때 0~100이외의 값이 입력된 경우, 처음 명령으로 돌아간다.

```
# changescore
Student ID:20180007
Mid/Final?mid
Input new score:147
#
```

시험 점수까지 올바르게 입력하였다면 중간고사 혹은 기말고사의 점수를 수정하여 출력한다.

```
# changescore
Student ID:20180007
Mid/Final?mid
Input new score:75
Student      Name      Mid      Final      Average Grade
-----
20180007     Kim Cheolsu    57       62       59.5      F
Score changed.
20180007     Kim Cheolsu    75       62       68.5      D
#
```

변경된 값을 show로 확인하면 다음과 같다.

```
# show
Student      Name      Mid      Final      Average Grade
-----
20180002     Lee Jieun     92       89       90.5      A
20180009     Lee Yeonghee  81       84       82.5      B
20180001     Hong Gildong  84       73       78.5      C
20180007     Kim Cheolsu   75       62       68.5      D
20180011     Ha Donghun    58       68       63.0      D
#
```


add를 입력하는 경우, 학생의 학번을 요구한다. 이미 존재하는 학생이라면 "ALREADY EXISTS."라는 에러 메시지를 출력한다.

```
# add
Student ID:20180001
ALREADY EXISTS
#
```

학생이 중복되지 않았다면 이름, 중간고사 점수, 기말고사 점수를 입력받아 등록하고 "Student added."라는 메시지를 출력한다.

```
# add
Student ID:20180021
Name:Kim Namkyu
Midterm Score:93
Final Score:95
Student added.
# add
Student ID:20180006
Name:Lee Sangsun
Midterm Score:77
Final Score:66
Student added.
#
```

입력한 결과를 show로 확인하면 다음과 같다.

```
# show
```

Student	Name	Mid	Final	Average	Grade
20180021	Kim Namkyu	93	95	94.0	A
20180002	Lee Jieun	92	89	90.5	A
20180009	Lee Yeonghee	81	84	82.5	B
20180001	Hong Gildong	84	73	78.5	C
20180006	Lee Sangsun	77	66	71.5	C
20180007	Kim Cheolsu	75	62	68.5	D
20180011	Ha Donghun	58	68	63.0	D

```
#
```

searchgrade를 입력하는 경우, 원하는 등급을 요구한다. A/B/C/D/F 이외의 값을 입력하였다면 처음 명령으로 돌아간다.

```
# searchgrade
Grade to search:E
#
```

입력한 등급에 해당하는 학생이 없는 경우, "NO RESULTS."라는 에러 메시지를 출력한다.

```
# searchgrade
Grade to search:F
NO RESULTS
#
```

입력한 등급에 학생이 있는 경우, 학생을 모두 출력한다.

```
# searchgrade
Grade to search:D
Student      Name      Mid      Final      Average Grade
-----
20180007     Kim Cheolsu    75       62       68.5      D
20180011     Ha Donghun     58       68       63.0      D
#
```

remove를 입력하였을 때, 학생 목록에 아무도 없는 경우 "List is empty"라는 오류 메시지를 출력한다.

```
# remove
List is empty
#
```

목록에 학생이 있는 경우에는 학번을 요구한다. 학번에 해당하는 학생이 없는 경우 "NO SUCH PERSON"이라는 에러 메시지를 출력한다.

```
# remove
Student ID:20180030
No Such Person
#
```

삭제하고자 하는 학생이 존재하는 경우, 해당 학생의 정보를 모두 삭제하고, "Student removed."라는 메시지를 출력한다.

```
# remove
Student ID:20180011
Student removed.
#
```

(다음장에 이어서)

삭제한 결과를 show로 확인하면 다음과 같다.

```
# show
Student      Name      Mid    Final  Average Grade
-----
20180021     Kim Namkyu    93     95     94.0    A
20180002     Lee Jieun     92     89     90.5    A
20180009     Lee Yeonghee  81     84     82.5    B
20180001     Hong Gildong  84     73     78.5    C
20180006     Lee Sangsun   77     66     71.5    C
20180007     Kim Cheolsu   75     62     68.5    D
# 
```

quit을 입력하는 경우, 현재까지 편집한 내용의 저장 여부를 묻는다. no를 입력하거나 잘못 입력한 경우, 처음 명령으로 돌아간다.

```
# quit
Save data?[yes/no]no
# quit
Save data?[yes/no]noooo
# 
```

yes를 입력하는 경우, 파일명을 물어본 후, 해당하는 파일명에 데이터를 저장한다. 프로그램 종료시, "프로그램 끝"이라고 출력한다.

```
# quit
Save data?[yes/no]yes
File name:newStudents
프로그램 끝 
```

5. 토론

과제를 수행하는 과정에서 여러가지 문제점과 보완해야 할 점을 도출할 수 있었다. 주요한 내용으로는 다음과 같다.

- 안정적으로 자료를 처리하기 위하여 txt파일을 열고 임시 list에 내용을 저장한 후, 파일을 닫았다. 이후 임시 list의 내용을 split하여 메인 list에 담는 과정을 진행하였다. 하지만 txt 파일의 내용이 적은 경우에는 임시 list를 활용하지 않고 바로 처리해도 오류가 발생하지 않으며, 코드를 간소화할 수 있다.
- changescore(x)를 작성하는 과정에서 시험의 종류가 mid일 때와 final일 때의 변경되는 값은 다르지만, 변경 과정 자체는 동일했다. 이 부분을 함수로 분리하여 적용하면 코드를 간소화 할 수 있었으나, 이미 정의된 def의 수가 많아 내용을 직접 작성하였다.
- for문 내부에서 다양한 if문이 반복적으로 실행되면서 코드의 가독성과 직관성이 저하되는 문제를 확인하였다. pass와 return None을 활용하여 흐름을 명확히 구분할 수 있었으나, 코드 간소를 위하여 해당 부분을 생략하였다.

6. 결론

해당 과제를 통하여 학생 성적이 포함된 파일로부터 성적 목록을 생성하고 이를 관리하는 성적 관리 프로그램을 제작하였다. 교육간에 학습한 내용들을 바탕으로 사용자의 요청에 따라 정보를 추가 · 수정 · 삭제할 수 있도록 구현해보았으며, 처리된 결과를 새로운 파일로 저장하여 채점자의 성적 관리 및 처리의 효율성을 향상시킬 수 있었다.

7. 개선방향

본 과제는 과제 수행을 목적으로 제작된 프로그램으로, 모든 사용자가 정의된 명령어를 이미 알고 있다는 전제 하에 작성되었다. 그러나 실제 환경에서 프로그램을 활용할 경우, 명령어가 사전에 정의되어 있지 않아 사용자가 혼란을 겪을 가능성이 있다. 따라서 명령어를 번호 형태로 제시하고, 사용자가 해당 번호를 입력하도록 개선한다면 사용 편의성을 높일 수 있다. 또한, 해당 프로그램은 입력 과정에서 오타가 발생할 때마다 초기 함수로 되돌아가는 불편함이 존재한다. 과제 조건상 이 과정을 그대로 두었지만, 오류 발생 시 다시 입력을 유도하고, 잘못 기입한 경우 이전 기입으로 돌아갈 수 있게 구현한다면 편의성을 높일 수 있다.